Grokking Under a Fixed Budget

One Long Run or Several Restarts?

Name: _____ Registration number: _____

1. Problem and clear objectives

A small neural network can fit its training examples before it performs reliably on unseen examples. Delayed improvement of this kind is called grokking [1]. Repeating a training experiment with different starting weights can produce different waiting times. The problem is therefore to measure that variability and ask how a limited training budget should be allocated.

Research question: With the same total number of training updates, is one long run or several shorter runs from independent initializations more likely to reach a generalization threshold?

Objectives: (i) measure time to generalization across repeated runs; (ii) compare capped learning times across weight-decay settings; (iii) estimate success probability by a budget; and (iv) use these estimates to predict fixed-budget restart outcomes.

Timing prediction already has related research [2], and restart schedules have established foundations [3]. The proposed contribution is a controlled empirical comparison in this small task. Neither grokking nor the restart probability identity is claimed as a new discovery.

2. Mathematical and statistical methods

Endpoint: T is the update at the third consecutive evaluation where training accuracy is at least 99% and unseen accuracy is at least 95%. Evaluations occur every 25 updates. Qualifying evaluations at 2,000, 2,025 and 2,050 therefore give T = 2,050. Memorization time is recorded separately from the training threshold.

Observation window: C = 30,000 updates. A run unfinished by C keeps an unknown T. Define U = min(T,C) for fully observed capped-time comparisons. Assigning U = C to an unfinished run does not imply it completed at C. A first confirmed event is also distinct from permanently high accuracy.

Statistical methods and implementation

2. Mathematical and statistical methods (continued)

Descriptive statistics: For n runs, the sample mean is sum(U_i)/n and sample variance is sum((U_i - mean U)^2)/(n-1). The median and standard deviation help describe typical waiting time and spread. Histograms reveal skewness that an average alone can hide.

Probability: Estimate F(t) = P(T ≤ t) by the fraction of all observed runs with a confirmed event by t. A success indicator is Bernoulli; independent equal-probability indicators give a binomial success count. Exact binomial confidence intervals quantify probability uncertainty. A 1,000-resample percentile bootstrap estimates uncertainty in mean U.

Comparisons: Welch tests compare pairs of group means, with Holm adjustment for multiple comparisons. One-way ANOVA tests H_0: the three population mean capped times are equal. Pearson/Spearman correlations describe associations; simple regression uses U = β_0 + β_1 log(weight decay) + error. Small samples and non-normal residuals require caution. Permutation sensitivity checks still require exchangeable errors.

Restarts: Give each of k independent runs B/k updates. If a single run succeeds with probability F(B/k), all fail with probability [1 - F(B/k)]^k. Hence P(at least one success) = 1 - [1 - F(B/k)]^k. This uses elementary independent-event probability; it is not a new theorem. Predictions estimated from the same cohort are correlated.

The supplied syllabus [4] supports random variables and moments (Module 1), Bernoulli/binomial outcomes (Module 2), inference (Module 3), correlation/regression (Module 4) and ANOVA (Module 5). Review 2 will cover candidate time distributions and expanded factorial analysis. Unsuitable syllabus tests are not forced onto this dataset.

3. Other methodology used

Python generates y = (a+b) mod 31 for all 961 ordered pairs, for example (30+4) mod 31 = 3. In this review, 576 pairs train the model and 385 disjoint pairs evaluate it. The split seed stays fixed; independent initialization seeds control starting weights. One full-batch update adjusts weights using the whole training subset.

A fixed small transformer uses embedding width 64, four attention heads and a 128-unit feed-forward layer. AdamW uses learning rate 0.003. Weight decay, which shrinks weights during optimization, takes values 0.3, 1 and 3. Nine separate pilot runs select decay 1 by the predefined rule; main group sizes are 6, 12 and 6. Python preserves traces, configuration, statuses and resumable checkpoints. Base R reads CSVs and performs analysis. CUDA was unavailable; this cohort used CPU.

Initial results from the frozen cohort

4. Demonstration of initial results

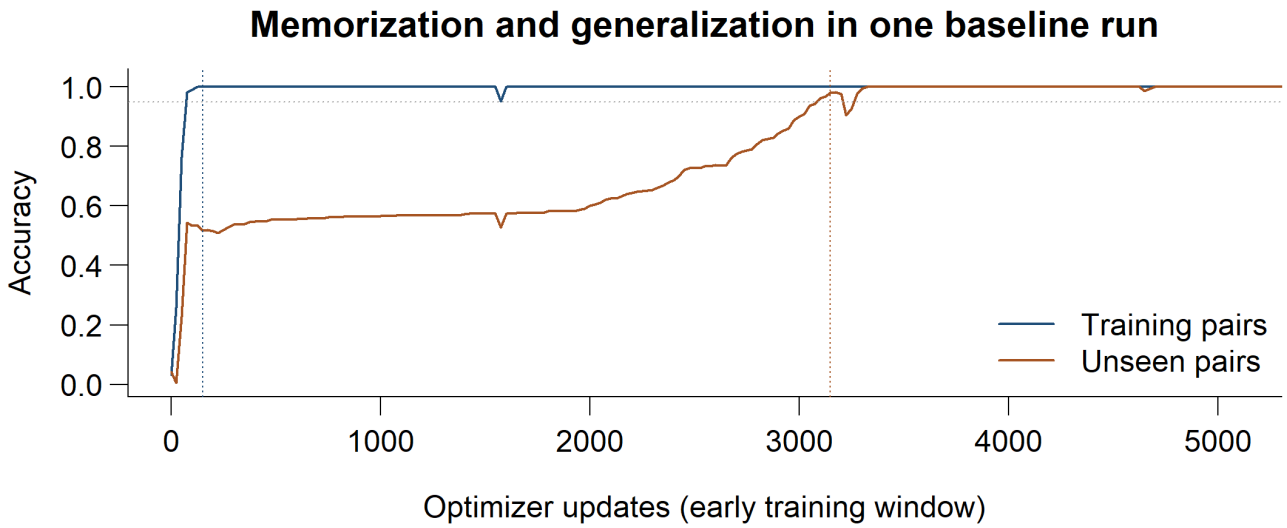


Figure 1. Training and unseen accuracy for a representative baseline run near its median event time. The early window shows a delay; the complete trace extends to 30,000 updates.

Weight decay	Runs / events	Mean U	SD U
0.3	6 / 6	13,137.5	4,593.4
1	12 / 12	6,475.0	7,041.4
3	6 / 6	1,887.5	889.8

Table 1. Capped times in optimizer updates; all groups use 60% training data. All 24 runs reached the threshold within the cap. Pilot runs are not included.

The baseline median is 4,275 updates, with observed range 1,875-27,175. Its mean exceeds its median, and its SD exceeds its mean, indicating substantial spread. The bootstrap 95% interval for mean U is approximately 3,579-10,615 updates. Baseline success by C is 12/12, with exact 95% interval 73.5%-100%; the sample does not establish certainty of population success.

The preliminary one-way ANOVA gives $p = 0.0077$ and permutation sensitivity $p = 0.0130$. Residual normality fails (Shapiro-Wilk $p \approx 0.000068$), so the comparison remains exploratory. Larger weight decay is associated with shorter capped time in this cohort, not established as universally optimal. Pearson r for log(decay) and U is -0.606; Spearman ρ is -0.741.

Eighteen of 24 main runs later dipped below 95% unseen accuracy. The endpoint measures first confirmation, not stable generalization. All results are conditional on one fixed data split.

Restart predictions and future work

4. Initial results (continued)

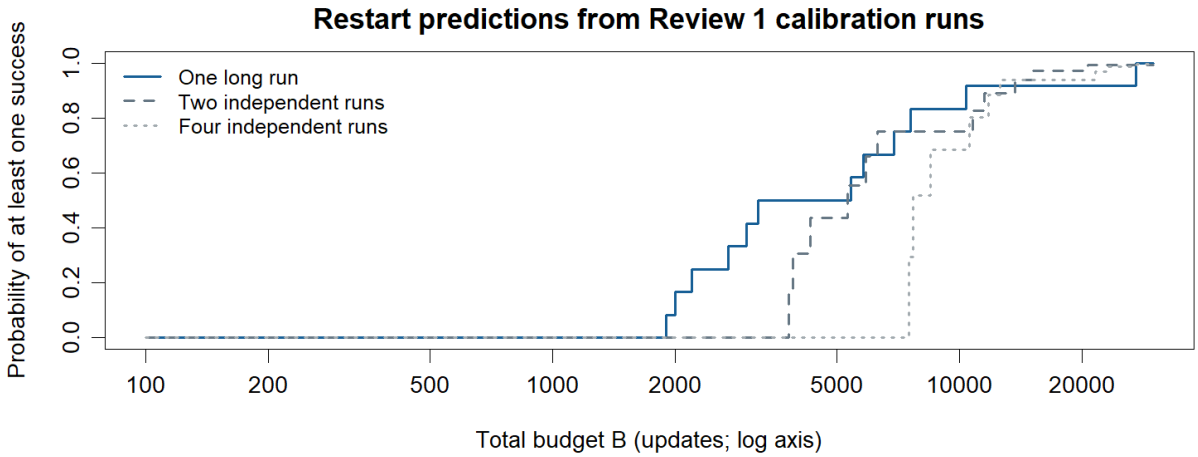


Figure 2. Restart success predicted from the 12 baseline runs. These predictions are unvalidated in the Review 1 cohort.

At total B = 30,000, predicted success is 100.0% for k=1, 99.3% for k=2 and 99.6% for k=4. The allocations are respectively 30,000, 15,000 and 7,500 updates per run. At B = 2,400, the predictions are 25%, 0% and 0%. A zero empirical estimate does not establish zero population probability. The generous full cap may conceal differences near the learning transition.

5. Future work and Review 2 coverage

The frozen Review 1 milestone covers the training/logging pipeline, resumption, repeated-run export, policy runner, preliminary R analysis and first report: approximately 65% of the weighted implementation checklist, not 65% of scientific certainty. Review 2 will cover the expanded six-cell study, candidate Exponential/Gamma/Weibull models, multiple regression, two-factor ANOVA and fresh-seed policy comparison.

Further research includes validating budgets near the learning transition, repeating the study with different data-split seeds, and measuring sustained rather than first-confirmed generalization. Runtime should be reported separately from logical update budgets. An institutional similarity score remains pending because no checker or acceptance threshold has been specified.

References

[1] Power et al. (2022). Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. https://arxiv.org/abs/2201.02177

[2] Truong et al. (2026), preprint. First-Passage Prediction of Grokking Delay: A Calibrated Law under AdamW with Causal Validation. https://arxiv.org/abs/2605.18845

[3] Streeter, Golovin and Smith (2007). Restart Schedules for Ensembles of Problem Instances. https://www.cs.cmu.edu/~dgolovin/papers/aaai07b.pdf

[4] BAMAT207 official theory and embedded-lab syllabus, version 1; supplied archive. [5] Project evidence: frozen Review 1 summary, raw run traces and base-R outputs.